

ZWaste Monitor AI

1. CORE OBJECTIVE

ZWaste Monitor AI must allow authorised users to register and manage municipalities and all major wastegenerating entities within those municipalities.

The initial system must manage:

1. Municipalities
2. Institutions
3. Businesses
4. Hospitals
5. Waste collection vehicles
6. Drivers
7. Waste collection routes
8. Waste records
9. Waste containers/bins
10. Waste reports
11. AI predictions
12. Alerts
13. Analytics
14. Future household waste records

The architecture must allow **household-level waste management to be activated in the future without rebuilding the system.**

Do NOT use the word “School” as a primary entity.

Use:

Institution

Institutions may later have different types, such as:

- School
- University
- College
- Government institution
- Public facility
- Private institution
- Other

2. MULTI-MUNICIPALITY ARCHITECTURE

The system must support multiple municipalities.

An administrator must be able to create a municipality using:

- Municipality number/ID
- Municipality name
- Province
- Region
- Country
- Address
- Geographic coordinates
- Contact information
- Municipality status

Each municipality must have its own data.

A user assigned to Municipality A must not automatically see Municipality B's private operational information.

Create proper multi-tenant data relationships.

3. MUNICIPALITY PROFILE

Each municipality must have a dedicated dashboard.

Display:

- Municipality name
- Municipality ID
- Total institutions
- Total businesses
- Total hospitals
- Total households when the future household module is activated
- Total waste generated
- Total waste collected
- Total uncollected waste
- Number of active bins
- Number of critical bins
- Number of trucks
- Number of active drivers
- Active collection routes
- Open waste complaints
- Current alerts
- AI risk score

-

- Waste-management performance score
-

4. INSTITUTION MANAGEMENT

Create an **Institutions** module.

The user must be able to:

- Register an institution
- Edit an institution
- Delete/deactivate an institution

-
-
- Search institutions
- Filter institutions
 - View institution history Each institution must contain:
 - Institution ID
 - Institution name
 - Institution type
 - Municipality
 - Area
 - Address
 - Latitude
 - Longitude
 - Contact information
 - Waste-generation profile
 - Collection frequency
 - Number of bins
 - Waste types
 - Current waste level
 - Historical waste records
 - Collection history
 - Status

Institution types must be configurable.

Do not hard-code the system to schools.

5. BUSINESS MANAGEMENT

Create a **Businesses** module.

Allow authorised users to register individual businesses.

Each business should contain:

- Business ID
- Business name
- Business type/category
- Municipality
- Area
- Address
- Latitude
- Longitude
- Contact information
- Estimated waste generation

-
-
- Actual recorded waste
- Waste types
- Number of containers
- Collection frequency
- Collection history
- Current waste level
- Status

Allow businesses to be classified by type, for example:

- Retail
- Restaurant
- Factory
- Office
- Warehouse
- Shopping centre
- Market
- Other

Make business categories configurable.

6. HOSPITAL MANAGEMENT

Create a dedicated **Hospitals** module.

Each hospital must contain:

- Hospital ID
- Hospital name
- Municipality
- Area
- Address
- Latitude
- Longitude
- Number of waste containers
- Waste generation
- Waste categories
- Collection frequency
- Collection history
- Current waste level
- Status

Because healthcare waste can require specialised handling, allow waste categories such as:

- General waste

-
-
- Recyclable waste
- Organic waste
- Medical/clinical waste
- Sharps
- Pharmaceutical waste
- Other regulated waste

Do not provide unsafe disposal instructions.

The system should simply record, classify and flag waste categories that require specialised handling.

7. FUTURE HOUSEHOLD MODULE

Build the database and software architecture so that **household-level waste monitoring can be enabled in the future**.

Do not require household data for the first version.

Create the data model and API structure now.

Future household records should support:

- Household ID
- Municipality
- Area
- Optional address/location
- Household waste generation
- Waste types
- Bin/container ID
- Collection schedule
- Collection history
- Recycling information
- Waste-generation history
- Alerts
- Service status

Privacy must be considered carefully.

Do not unnecessarily collect personal information.

The system must support anonymous or pseudonymous household identifiers where possible.

Create the household module as a feature that can be activated later.

8. WASTE RECORDING SYSTEM

Create a universal **Waste Records** system.

Every waste record should contain:

- Record ID
- Municipality
- Entity type
- Entity ID
- Date
- Time
- Waste type
- Waste quantity
- Measurement unit
- Collection status
- Collection vehicle
- Driver
- Route
- Source
- Sensor/manual/API
- Notes

Entity types should include:

- Institution
- Business
- Hospital
- Household (future)
- Public area
- Other

Allow waste quantities to be recorded in appropriate units such as:

- kilograms
- tonnes
- litres where appropriate

9. WASTE TYPES

Create configurable waste categories.

Initial categories:

- General waste
- Recyclable
- Organic

- Plastic
- Paper
- Glass
- Metal
- Electronic waste
- Construction waste
- Medical/clinical waste
- Hazardous/regulated waste
- Other

Administrators must be able to add new categories.

10. BIN AND CONTAINER MANAGEMENT

Create a **Bins & Containers** module.

Each container should have:

- Bin ID
 - Municipality
 - Entity type
 - Entity ID
 - Location
 - Capacity
 - Current fill percentage
 - Waste type
 - Sensor ID
 - Battery level
 - Temperature when available
 - Sensor status
 - Last reading
 - Last collection
 - Predicted overflow time
 - Priority
 - Status Statuses:
 - EMPTY
 - LOW
 - NORMAL
 - HIGH
 - CRITICAL
 - OFFLINE
-

•

11. SENSOR DATA

Create an API capable of receiving IoT data.

Example data:

- Bin ID
- Sensor ID
- Fill percentage
- Temperature
- Battery level
- GPS/location
- Sensor status
- Timestamp Validate incoming data.

Reject invalid readings.

Store historical readings.

Allow the system to identify unusual sensor behaviour.

12. ZWASTE MONITOR AI ENGINE

Create a dedicated Python AI/analytics layer.

The AI must analyse:

- Historical waste
- Current waste levels
- Collection frequency
- Bin fill rates
- Entity type
- Municipality
- Geographic area
- Vehicle availability
- Truck capacity
- Collection history

-

Seasonal patterns

- Day-of-week patterns
- Reported waste problems The AI must produce:
- Waste forecasts
- Overflow predictions
- Collection priorities
- Route recommendations
- Vehicle recommendations
- Waste-generation trends
- Anomaly detection
- Municipal performance analysis
- Resource recommendations

13. WASTE PREDICTION

Create a prediction engine that estimates when a bin/container or waste-generating entity may reach a critical level.

For each prediction display:

Prediction

Confidence/uncertainty

Reason

Recommended action

Example:

“Container GZ-104 is predicted to reach 90% capacity within approximately 6 hours. The prediction is based on recent fill-rate trends and historical collection data. Recommended action: include the container in the next high-priority collection route.”

Do not present simulated predictions as real municipal facts.

Clearly distinguish:

- Real sensor data
- Manual data
- Simulated data
- AI predictions

-

14. AI DECISION ENGINE

Create reusable Python services/functions such as:

- calculate_waste_generation()
- predict_overflow()
- calculate_collection_priority()
- detect_anomaly()
- recommend_collection()
- recommend_vehicle()
- optimise_route()
- calculate_municipality_score()
- generate_alert()
- generate_municipality_report()
- forecast_waste()

Separate the AI logic from the user interface.

15. MUNICIPAL WASTE INTELLIGENCE

ZWaste Monitor AI must be able to analyse an entire municipality.

For example, it should be able to determine:

- Which area produces the most waste?
- Which entity type produces the most waste?
- Which institutions need collection?
- Which businesses generate unusually high waste?
- Which hospitals require scheduled collection?
- Which containers are approaching overflow?
- Which trucks are available?
- Which trucks are overloaded?
- Which routes are inefficient?
- Which areas experience repeated missed collections?
- Where are waste problems increasing?
- How much waste was generated?
- How much was collected?
- How much remains uncollected?

16. AI MUNICIPALITY SCORE

Create a dynamic **Waste Management Performance Score**.

The score should consider factors such as:

- Collection efficiency

- Overflow frequency
- Missed collections
- Waste recovery/recycling
- Vehicle utilisation
- Response time
- Uncollected waste
- Reported problems
- Sensor availability

Display the score with an explanation.

Do not represent the score as an official municipal government rating.

17. TRUCK/FLEET MANAGEMENT

Create a **Fleet Management** module.

Each waste truck must have:

- Vehicle ID
- Registration number
- Vehicle type
- Capacity
- Municipality
- Driver
- Driver ID
- Current location
- Current route
- Vehicle status
- Fuel level or battery level
- Mileage
- Maintenance status
- Last maintenance
- Next maintenance
- Current waste load
- Maximum load
- Collection performance

Statuses:

- AVAILABLE
- COLLECTING
- RETURNING
- MAINTENANCE
- OFFLINE
- OUT_OF_SERVICE

18. DRIVER MANAGEMENT

Create a **Drivers** module.

Store only information necessary for system operations.

Include:

- Driver ID
- Name
- Assigned municipality
- Assigned vehicle
- Contact information where authorised
- Employment/operational status
- Current route
- Collection history
- Performance metrics

Do not expose unnecessary personal information.

19. ROUTE OPTIMISATION

Create a route optimisation engine.

The system should consider:

- Bin/container priority
- Waste level
- Geographic distance
- Vehicle capacity
- Number of stops
- Collection deadlines
- Vehicle availability
- Driver assignment
- Waste type requirements Generate recommended collection routes.

Display:

- Route ID
- Vehicle
- Driver
- Stops
- Estimated distance
- Estimated duration
- Expected waste load
- Priority
- Status

If real road-routing data is unavailable, use a simulated distance model and clearly label it as a prototype estimate.

20. ALERTS

Automatically create alerts for:

- Critical bin
- Predicted overflow
- Missed collection
- Delayed collection
- Sensor offline
- Low sensor battery
- Vehicle maintenance
- Vehicle capacity exceeded
- Unusual waste increase
- Repeated waste complaints
- Route problems Alert levels:
 - INFORMATION
 - LOW
 - MEDIUM
 - HIGH
 - CRITICAL

Allow authorised users to acknowledge and resolve alerts.

21. COMMUNITY WASTE REPORTING

Create a public/authorised **Waste Problem Reporting** interface.

Users should be able to report:

- Overflowing bin
- Illegal dumping

- Missed collection
 - Damaged container
 - Waste accumulation
 - Illegal waste site
 - Other waste problem Allow:
-
- Location
 - Description
 - Category
 - Severity
 - Date/time
 - Optional image upload in future Do not require unnecessary personal information.

Statuses:

- NEW
- RECEIVED
- INVESTIGATING
- ASSIGNED
- RESOLVED
- CLOSED

22. AI ASSISTANT INTEGRATION

Integrate an intelligent assistant into ZWaste Monitor AI.

Assistant name:

Rhooms

Rhooms must be able to answer natural-language questions using authorised system data.

Examples:

“Which municipality has the highest waste generation?”

“Which institutions require collection today?”

“Which businesses have increased their waste production?”

“How many hospitals are registered?”

“How many trucks are currently available?”

“Which bins are predicted to overflow?”

“Why is this area receiving a high collection priority?”

“Generate a waste-management report for this municipality.”

“Compare waste generation between two municipalities.”

Rhooms must explain its reasoning using available data.

Do not invent information.

If information is unavailable, say so.

23. DASHBOARD

Create a professional dashboard.

Display:

- Total municipalities
 - Total institutions
 - Total businesses
 - Total hospitals
 - Future household count
 - Total bins
 - Critical bins
 - Total trucks
 - Active trucks
 - Total waste generated
 - Total waste collected
 - Uncollected waste
 - Recycling/recovery percentage
 - Open alerts
 - Open complaints
 - AI predictions Add charts for:
 - Daily waste
 - Weekly waste
 - Monthly waste
 - Waste by entity type
 - Waste by municipality
 - Waste by area
 - Waste type
 - Collection efficiency
 - Fleet utilisation
-

24. MAP

Create a geographic monitoring interface.

Display:

- Municipalities
- Waste-generating entities
- Bins
- Collection vehicles
- Waste reports
- Critical locations
- Collection routes Allow filtering by:
 - Municipality
 - Entity
 - Waste type
 - Alert level
 - Bin status
 - Vehicle status

If an external mapping service requires an API key, use environment variables.

Do not hard-code API keys.

If no map API is available, provide a functional simulated map view.

25. MOBILE AND CROSS-PLATFORM SUPPORT

The ZWaste Monitor AI backend must be independent from the frontend.

Create APIs so future applications can connect.

Supported clients should include:

- Web
- Android
- iOS
- GateZ hardware
- IoT devices
- Other authorised applications

Do NOT build the core intelligence in a way that only works inside the web application.

The AI engine and database must be accessible through secure APIs.

26. API-FIRST ARCHITECTURE

Create documented API endpoints such as:

GET /api/municipalities

POST /api/municipalities

GET /api/municipalities/{id}

GET /api/institutions

POST /api/institutions

GET /api/businesses

POST /api/businesses

GET /api/hospitals

POST /api/hospitals

GET /api/bins

POST /api/bins

GET /api/vehicles

POST /api/vehicles

GET /api/drivers

POST /api/drivers

GET /api/waste-records

POST /api/waste-records

GET /api/predictions

GET /api/alerts

GET /api/routes

POST /api/routes

POST /api/sensor/readings

POST /api/reports

POST /api/ai/analyse

POST /api/ai/query

Design the API so Android and iOS applications can use the same backend.

27. GATEZ HARDWARE INTEGRATION

The architecture must support future GateZ hardware.

Target architecture:

GATEZ SENSOR GATEZ DEVICE HTTP/MQTT ZWaste Monitor AI API DATABASE AI ENGINE RHOOMS WEB /
ANDROID / iOS

The software must not pretend that the hardware exists yet.

Create a **GateZ Device Simulator**.

The simulator must allow the developer to simulate:

- Fill level
- Temperature
- Battery
- Sensor status
- Location
- Timestamp

When simulated data is submitted, it must enter the same processing pipeline as future real sensor data.

28. DATABASE

Use a relational database.

Use SQLite for the initial prototype.

Structure the application so it can later migrate to PostgreSQL.

Create tables/models for:

- Users
- Roles
- Municipalities
- Institutions
- Businesses
- Hospitals
- Households
- Bins
- Sensors
- SensorReadings
- Vehicles
- Drivers
- Routes
- RouteStops
- WasteRecords
- Predictions
- Alerts
- WasteReports
- AuditLogs

Create proper relationships and indexes.

The Household model may exist but remain inactive in the initial UI.

29. AUTHENTICATION AND SECURITY

Implement secure authentication.

Roles:

SUPER ADMIN

Full system access.

MUNICIPAL ADMIN

Access to assigned municipality.

OPERATOR

Can manage operational waste data.

DRIVER

Can see assigned routes and collection tasks.

ANALYST

Can access analytics and reports.

VIEWER

Read-only access.

Users must only access information they are authorised to see.

Passwords must never be stored as plain text.

Use environment variables for:

- AI API keys
- Database credentials
- Secret keys
- External service credentials

30. AUDIT LOGGING

Record important system actions.

Examples:

- User login
- Record created
- Record modified
- Record deleted • Alert resolved
- Route changed
- Vehicle assigned
- AI analysis requested Store:

- User
- Action
- Entity
- Timestamp
- Relevant information

31. DATA QUALITY

The system must distinguish between:

REAL DATA

MANUAL DATA

SIMULATED DATA

AI-GENERATED PREDICTION

Every important record should identify its source.

Do not combine simulated information with real information without clearly labelling it.

32. REPORT GENERATION

Allow authorised users to generate reports.

Reports should include:

- Municipality overview
 - Waste generation
 - Waste collection
 - Critical locations
 - Vehicle performance
 - Collection performance
 - Waste trends
 - AI predictions
 - Alerts
 - Recommendations Allow export to:
 - CSV
 - PDF
 - Excel where practical
-

33. AI RECOMMENDATION SYSTEM

The AI should produce recommendations such as:

Collection recommendation

“Collect these 14 containers within the next 8 hours.”

Fleet recommendation

“Three additional vehicles may be required based on predicted collection demand.”

Area recommendation

“Area X shows an increasing waste-generation trend.”

Maintenance recommendation

“Vehicle GZ-TRUCK-07 should be scheduled for maintenance.”

Operational recommendation

“Move collection capacity towards Area Y due to increased predicted demand.” Every recommendation must include:

- Recommendation
 - Reason
 - Supporting data
 - Priority
 - Confidence/uncertainty
 - Suggested action
-

34. FUTURE SCALABILITY

Design the system so it can later support:

- Thousands of municipalities
- Millions of waste records
- Large IoT sensor networks
- Household-level monitoring
- Real-time truck tracking
- Computer vision
- Image-based waste classification
- Advanced machine learning
- Predictive maintenance
- Carbon/emissions analytics
- Recycling optimisation
- Smart-bin hardware
- Edge AI
- Mobile applications

Do not over-engineer the first prototype, but ensure the architecture can grow.

35. TECHNOLOGY

Use:

- Python
- Reflex

- FastAPI or Reflex-compatible API architecture
- SQLite initially
- PostgreSQL-ready architecture
- Pandas
- NumPy
- Scikit-learn where useful
- Appropriate charting library
- Secure authentication
- REST APIs
- MQTT integration architecture for future IoT Use modern Python practices.

Keep AI, database, API and UI components modular.

36. USER INTERFACE

Create these main navigation sections:

1. Dashboard
2. Municipalities
3. Institutions
4. Businesses
5. Hospitals
6. Households — Future/Disabled Initially
7. Bins & Containers
8. Waste Records
9. AI Predictions
10. Routes
11. Fleet
12. Drivers
13. Alerts
14. Waste Reports
15. Analytics
16. Rhooms AI
17. GateZ Device Simulator
18. Administration

Use a modern smart-city design.

Make the application responsive for desktop, tablet and mobile screens.

37. DEMO DATA

Create realistic simulated demonstration data.

Include multiple municipalities and realistic South African geographic examples.

However:

Do not claim that simulated data represents actual City of Johannesburg municipal statistics.

Clearly label demo information:

DEMO DATA

The system should be immediately usable after installation.

38. DEMONSTRATION WORKFLOW

Create a complete demonstration workflow.

Example:

1. Administrator creates a municipality.
2. Administrator registers institutions.
3. Administrator registers businesses.
4. Administrator registers hospitals.
5. Administrator registers bins.
6. Administrator registers trucks.
7. Administrator assigns drivers.
8. Waste data is recorded.
9. Sensor simulator sends readings.
10. AI analyses the data.
11. Critical bins are identified.
12. Alerts are generated.
13. Routes are recommended.
14. Vehicles are assigned.
15. Rhooms explains the situation.
16. Analytics display the results.
17. A municipal report can be generated.

The complete workflow must function in the prototype.

39. IMPORTANT ETHICAL AND OPERATIONAL RULES

ZWaste Monitor AI is a decision-support system.

Do not automatically make high-impact operational decisions without appropriate human oversight.

AI recommendations should be presented as recommendations.

Clearly distinguish predictions from confirmed information.

Do not claim affiliation with the City of Johannesburg or any South African municipality unless authorised integrations actually exist.

Do not fabricate municipal statistics.

Do not expose private information unnecessarily.

Design with privacy, security and accountability in mind.

40. CODE ORGANISATION

Use a clean modular structure similar to: `zwaste_monitor_ai/`

```
app/  
  pages/  
  components/  
  models/  
  services/  
  ai/  
  api/  
  auth/  
  database/  
  analytics/  
  utils/  
  
data/  
tests/  
assets/  
  
requirements.txt  
.env.example  
README.md  
rxconfig.py
```

Adapt this to the current Reflex project structure where necessary.

41. TESTING

Create tests for:

- Municipality creation
- Institution creation
- Business creation
- Hospital creation

- Waste recording
 - Bin readings
 - AI prediction
 - Alert generation
 - Route calculation
 - Vehicle assignment
 - Authentication
 - API validation
-

42. README

Create a comprehensive README containing:

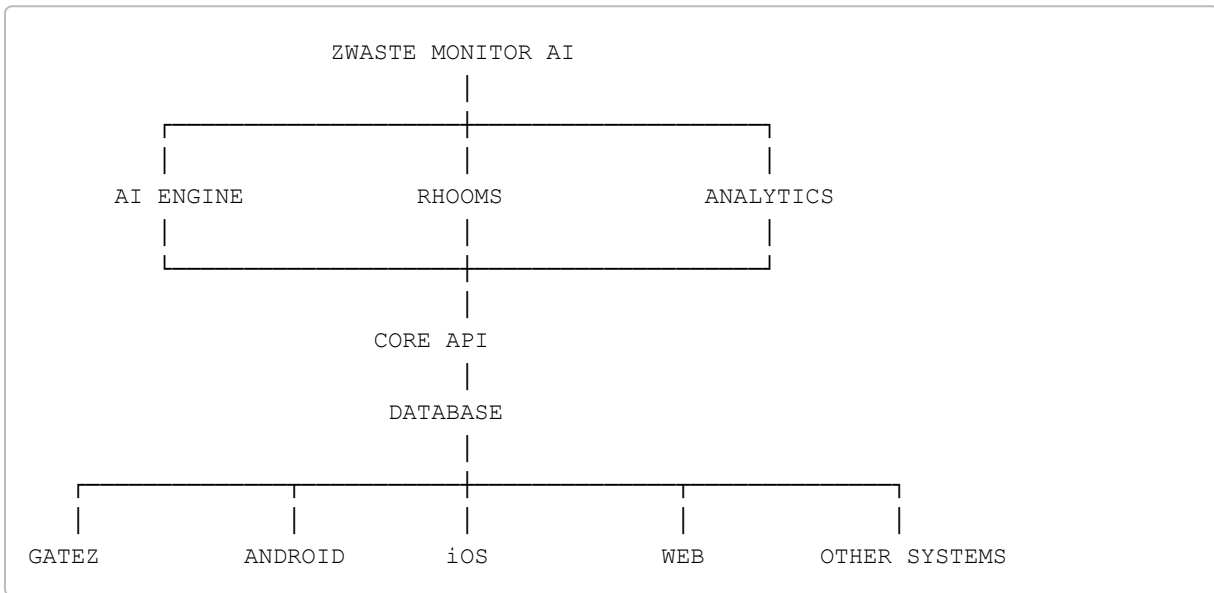
- What ZWaste Monitor AI is
- System architecture
- Features
- Installation
- Environment variables
- Database setup
- Running locally
- Demo mode
- AI configuration
- API documentation
- GateZ integration
- Android integration
- iOS integration
- Future household integration
- Deployment instructions
- Testing instructions

Explain exactly where the developer can find:

- AI engine
 - Rhooms assistant
 - Database models
 - API
 - Dashboard
 - Authentication
 - GateZ simulator
 - Household module
-

43. FINAL ARCHITECTURE

The final architecture should conceptually be:



HARDWARE APP APP DASHBOARD / IoT SENSORS

The central principle is:

ZWaste Monitor AI is the intelligence and data layer.

GateZ is one hardware platform that can connect to it, not the only platform.

44. FINAL REQUIREMENT

Build the first version as a genuinely functional prototype.

Do not produce merely:

- A landing page
- Static screens
- Fake buttons
- Placeholder dashboards
- Hard-coded AI responses

- Create actual:
- Database operations
 - CRUD functionality
 - AI analysis
 - Predictions
 - Alerts
 - API endpoints
 - Demo sensor data
 - Route calculations
 - Fleet management

- Municipality management
- Institution management
- Business management
- Hospital management
- Future household architecture
- Rhooms assistant
- Analytics
- Authentication

Prioritise functionality, maintainability and a clear architecture.

The final application should demonstrate how **ZWaste Monitor AI can become the central intelligence platform for a future smart waste-management ecosystem**, with GateZ hardware, mobile applications, web applications and authorised external systems all communicating through the same secure backend.